

Unity Catalog and Lineage

INTRODUCTION TO DATABRICKS LAKEHOUSE



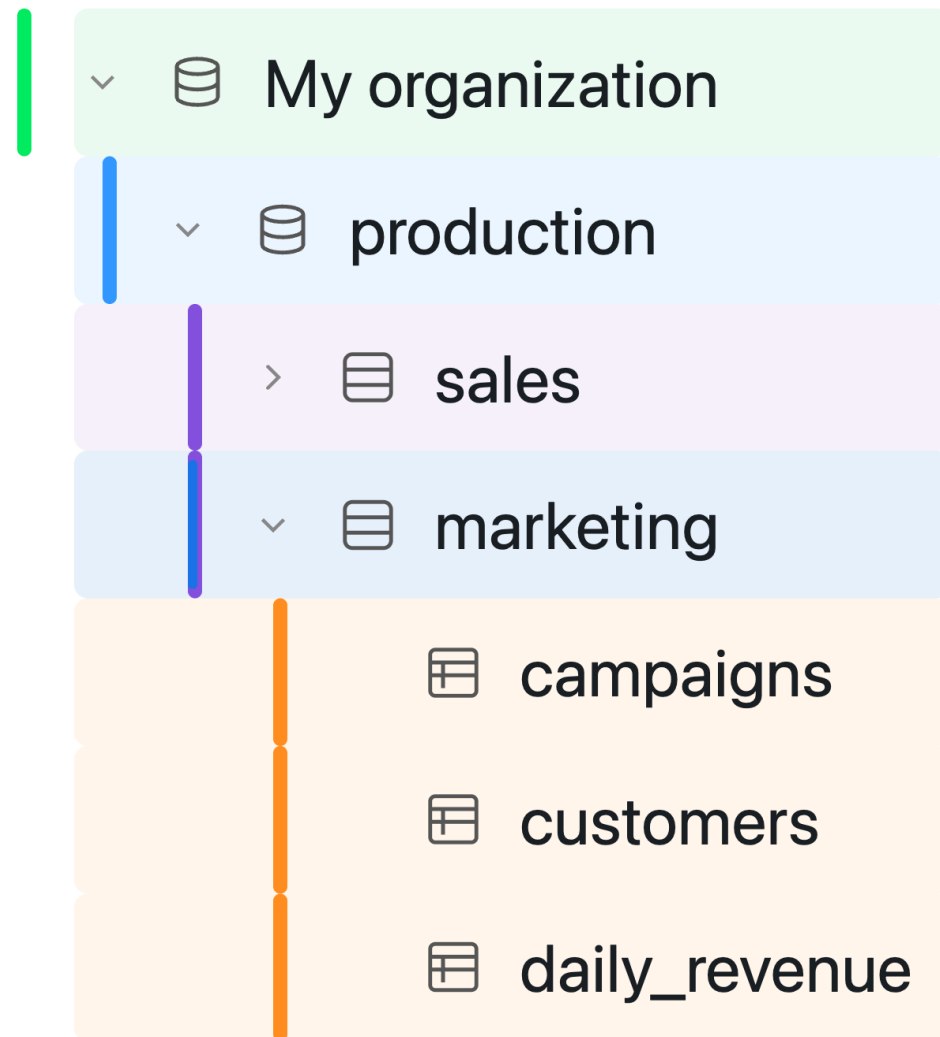
Gang Wang
Senior Data Scientist

Why governance matters

- Hundreds of tables across multiple teams
- "Can the marketing team see customer financial data?"
- "This report looks wrong - where does the underlying data actually come from?"
- **Unity Catalog** centralizes the answers



The Unity Catalog hierarchy



- **Metastore** - one per account, top-level container
- **Catalogs** - like "production" or "development"
- **Schemas** - group related objects (e.g. "sales")
- Contains **tables, views, and functions**

Access control

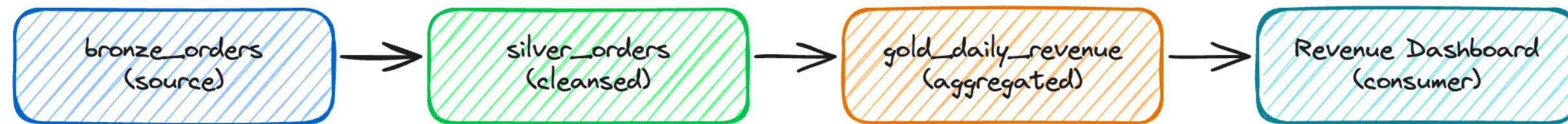
```
-- Grant read access to a schema
GRANT SELECT
ON SCHEMA production.sales
TO `analytics_team`;

-- Revoke table-level access
REVOKE SELECT
ON TABLE production.sales.customers
FROM `intern_group`;
```

- Permissions at **every level** of the hierarchy
- Grant `SELECT` , `MODIFY` , `CREATE` on catalogs, schemas, or tables
- Inherited - grant on a catalog applies to all schemas within it

Data lineage

Where did this data come from?



- **Automatic** tracking of table-to-table relationships as queries run
- Trace **upstream** sources and **downstream** consumers

Lineage in practice

- **Upstream** - trace where data comes from
- **Downstream** - see what depends on this table
- **Impact analysis** - if I change this column, what breaks?
- **Debugging** - a report shows wrong numbers, trace back to the source



Summary

- **Unity Catalog** - centralized governance for all data assets
- **Hierarchy** - metastore → catalog → schema
- **Access control** - SQL grants at every level, inherited downward
- **Lineage** - automatic tracking of data flow, upstream and downstream

Let's practice!

INTRODUCTION TO DATABRICKS LAKEHOUSE

Delta Sharing fundamentals

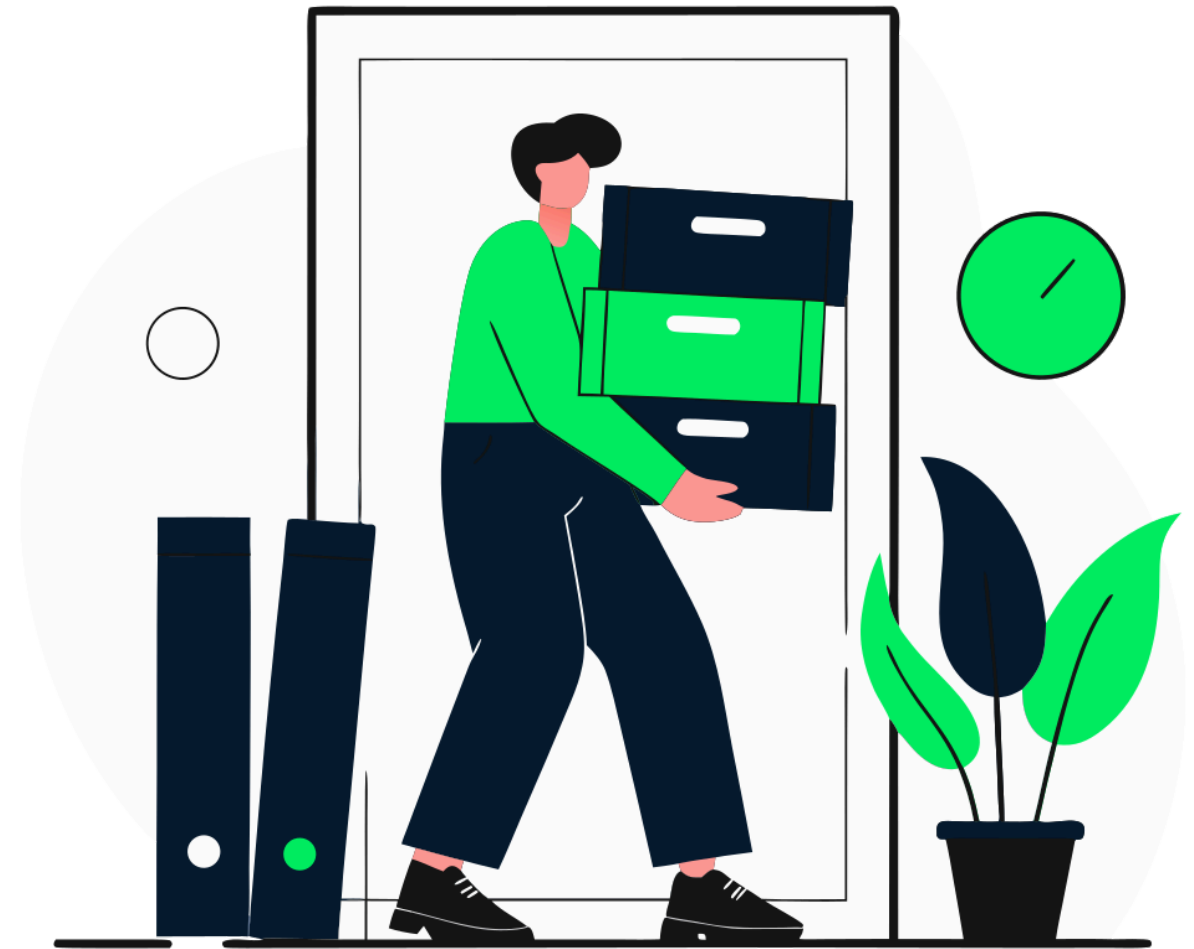
INTRODUCTION TO DATABRICKS LAKEHOUSE



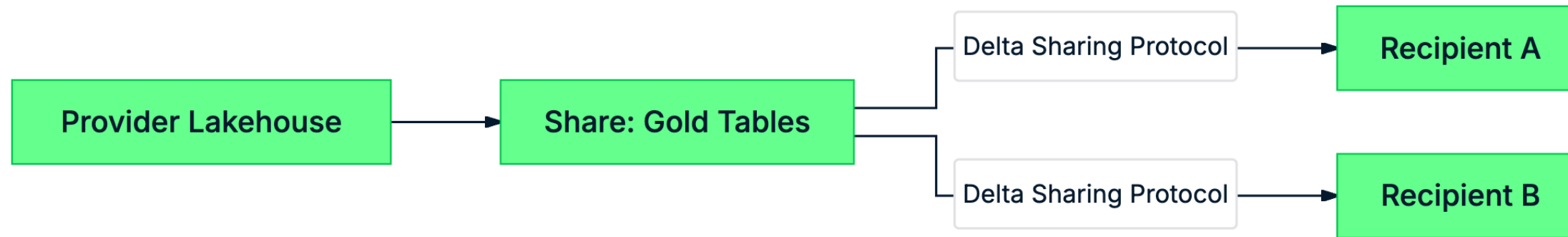
Gang Wang
Senior Data Scientist

The data sharing problem

- Traditional sharing: **export**, email, upload
- Data goes **stale** immediately
- **Multiple copies** across organizations
- No control once the file leaves your hands



What is Delta Sharing?



- **No copies** - recipients query live data through the Delta Sharing protocol
- **Revoke instantly** - remove access with a single command

Creating a share

```
-- Create a share
CREATE SHARE partner_data;

-- Add a gold table to the share
ALTER SHARE partner_data
ADD TABLE production.sales
    .gold_daily_revenue;
```

- A **share** is a named collection of tables
- Add tables from any catalog in Unity Catalog
- Typically share **gold layer** data - clean, aggregated, business-ready

Adding recipients

```
-- Create a recipient
CREATE RECIPIENT acme_corp;

-- Grant the recipient access
GRANT SELECT ON SHARE partner_data
TO RECIPIENT acme_corp;
```

- A **recipient** represents an organization or person
- They receive an **activation link** to set up access
- You control which shares each recipient can see

The library card analogy

- Instead of photocopying a book (exporting data), you issue a **library card** (a share)
- The recipient reads the **original in real time**
- You can **revoke the card** at any time



Summary

- **Delta Sharing** provides live, read-only access to data - no copies
- Create a **share**, add gold-layer tables, grant access to **recipients**
- Recipients query data in real time through the **Delta Sharing protocol**
- **Revoke access** instantly when the relationship ends

Let's practice!

INTRODUCTION TO DATABRICKS LAKEHOUSE

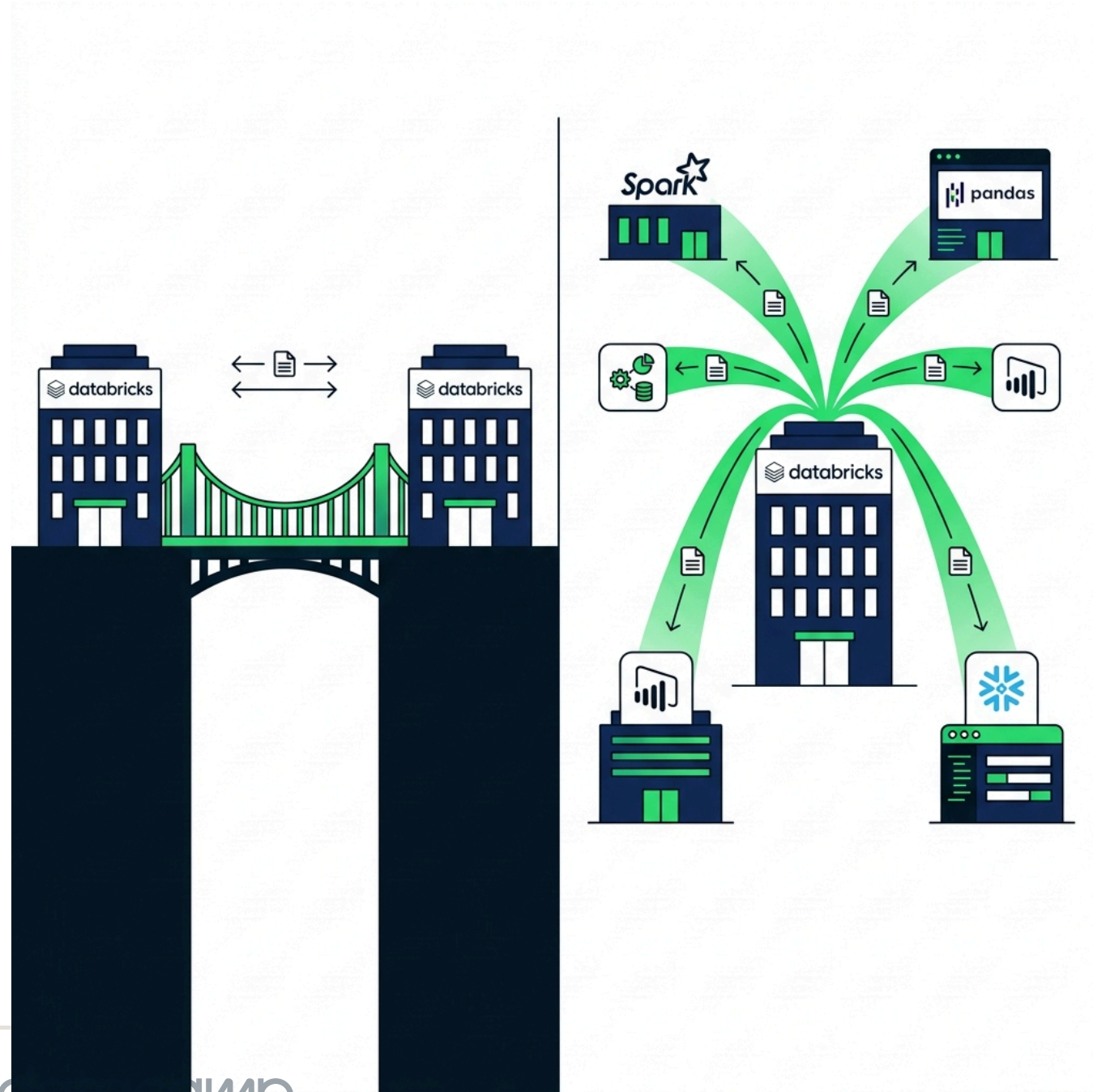
Delta Sharing: types and trade-offs

INTRODUCTION TO DATABRICKS LAKEHOUSE



Gang Wang
Senior Data Scientist

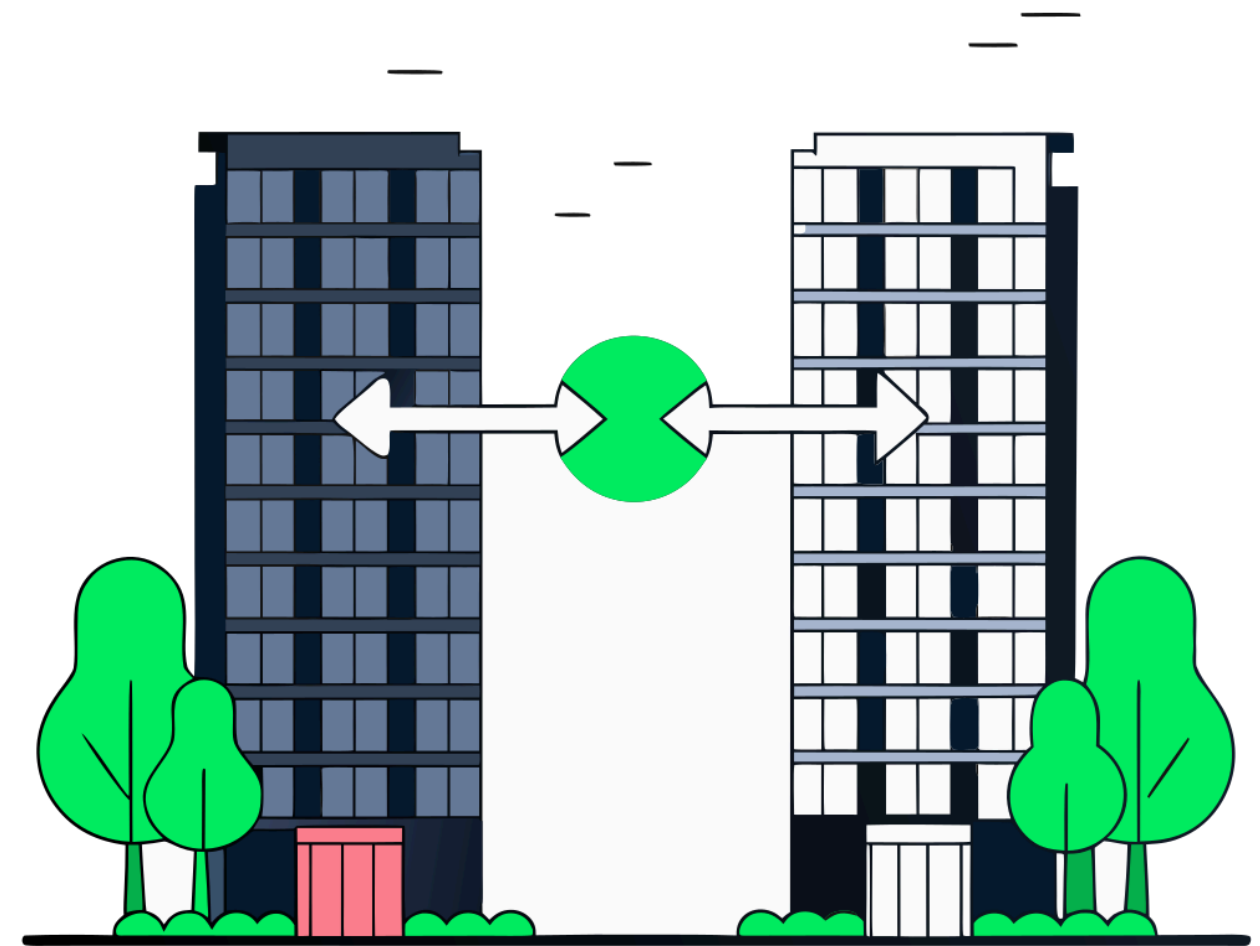
Two approaches to sharing



- **Native sharing** - Databricks to Databricks
 - Seamless Unity Catalog integration
 - Full **governance** on both sides
- **Open protocol** - any platform
 - Spark, pandas, Power BI, Snowflake
 - **More setup**, but **wider reach**

Databricks-native sharing

- Both parties are **Databricks** customers
- Shared data appears in recipient's **Unity Catalog**
- Full **governance** on both sides
- **Minimal setup** - just create the share and grant access



Open protocol sharing

```
# Recipient's code (using pandas)
import delta_sharing

profile = "config.share"
client = delta_sharing.SharingClient(
    profile
)
tables = client.list_all_tables()
df = delta_sharing.load_as_pandas(
    f"{profile}.share.schema.table"
)
```

- Recipient can use **any platform**
- Recipient installs a **Delta Sharing client**
- Spark, pandas, Power BI, Snowflake, Tableau

Cost considerations

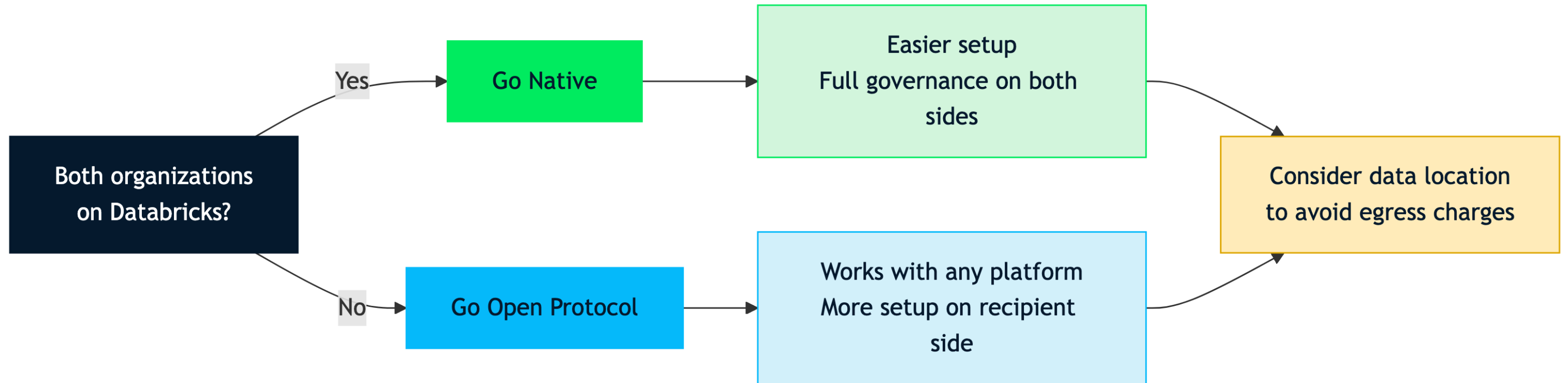
- **Same region** - typically no egress charges
- **Cross-region** - cloud provider charges for data transfer
- **Cross-cloud** (e.g., Azure to AWS) - highest egress fees
- Keep shared data near your main recipients

Same Region - Free or cheap

Cross-Region - Egress charges

Cross-Cloud - Highest fees

Choosing the right approach



Summary

- **Native sharing** - Databricks-to-Databricks, seamless, full governance
- **Open protocol** - any platform, broader reach, more setup
- **Egress charges** increase with cross-region and cross-cloud sharing
- Choose based on the recipient's platform and data location

Let's practice!

INTRODUCTION TO DATABRICKS LAKEHOUSE

Lakehouse Federation

INTRODUCTION TO DATABRICKS LAKEHOUSE



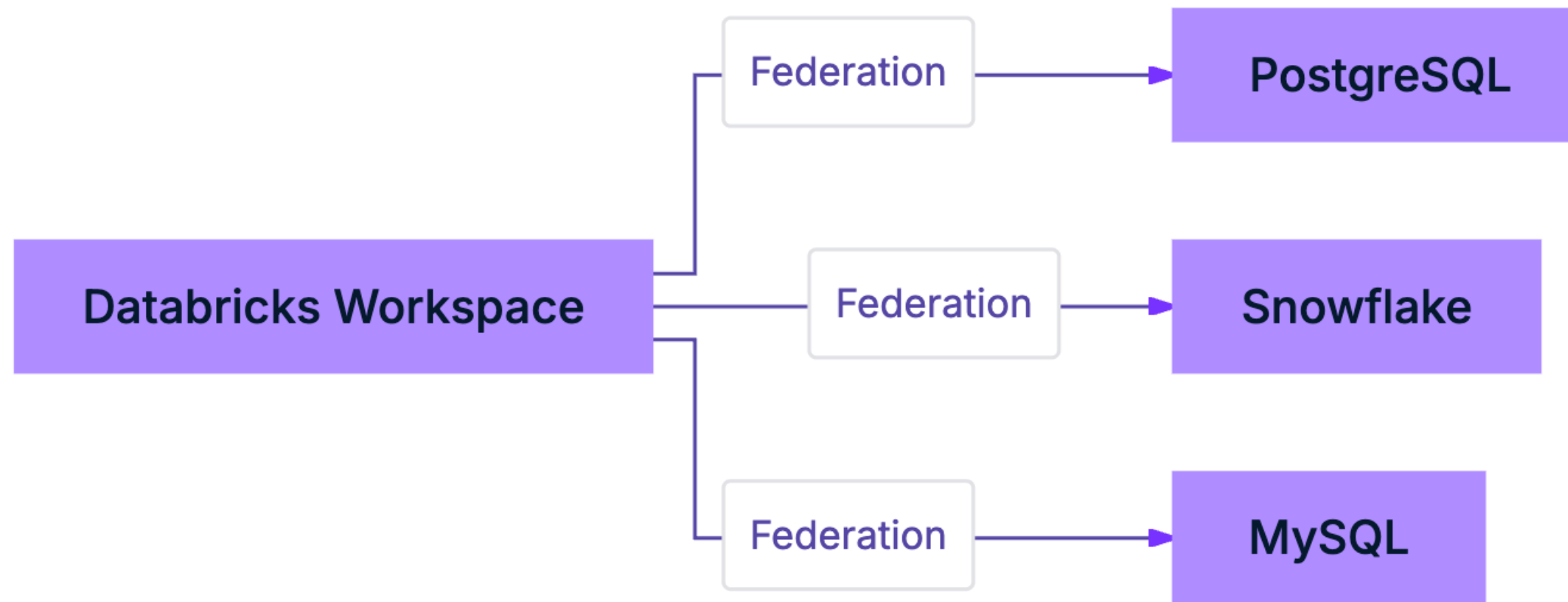
Gang Wang
Senior Data Scientist

The problem: data silos

- Data spread across **multiple systems**
- PostgreSQL, MySQL, Snowflake, SQL Server
- Moving data is **slow** and creates **copies**
- You need answers that span **all** your data



What is Lakehouse Federation?



Setting up a connection

```
-- Create a connection
CREATE CONNECTION partner_pg
TYPE postgresql
OPTIONS (
  host 'db.partner.com',
  port '5432',
  user 'reader',
  password secret('scope', 'key')
);
```

```
-- Create a foreign catalog
CREATE FOREIGN CATALOG partner_db
USING CONNECTION partner_pg;
```

```
-- Query the external table
SELECT *
FROM partner_db.public.orders
LIMIT 10;
```

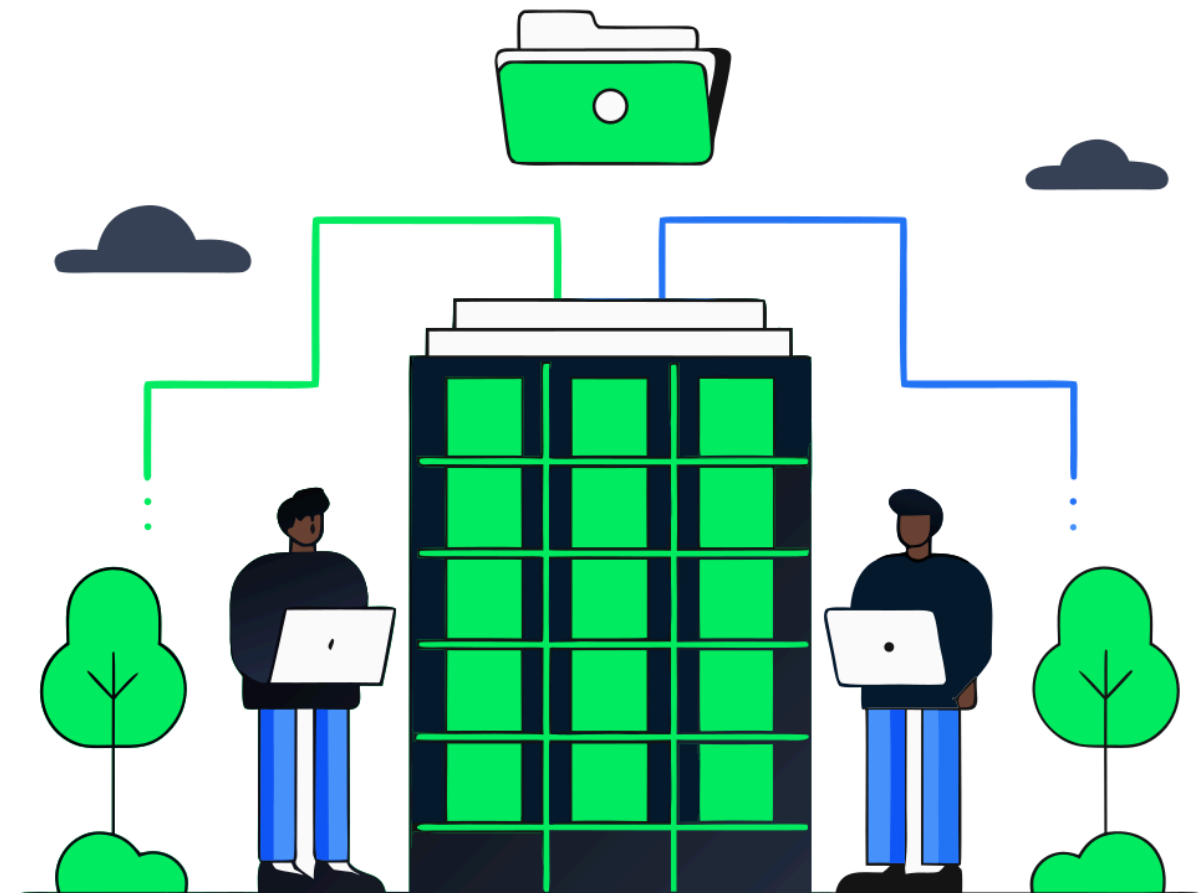
When to federate vs. when to ingest

	Federate	Ingest
Access	Real-time	Batch copy
Query volume	Low	High
Latency	Source-dependent	Low
Compliance	No copies needed	Data moves
Transforms	Limited	Full medallion

- **Federate** for real-time, low-volume, or compliance-restricted access
- **Ingest** for high-volume, repeated, performance-critical workloads

Federation in Unity Catalog

- Federated tables appear in the **Unity Catalog** hierarchy
- Same **access control** and **lineage** as local tables
- Join federated tables with local lakehouse tables in a single query
- Supported sources: PostgreSQL, MySQL, Snowflake, SQL Server, and more



Summary

- **Lakehouse Federation** queries external data without copying it
- Set up a **connection**, create a **foreign catalog**, query with standard SQL
- **Federate** for real-time, low-volume, or compliance-restricted access
- **Ingest** for high-volume, repeated, performance-critical workloads

Let's practice!

INTRODUCTION TO DATABRICKS LAKEHOUSE

